HackenProof

Expert crowdsourced security provider

FINAL REPORT

Private Security Audit

for **Multyr**

PREPARED BY

HackenProof

REPORT CREATED

07.08.2026

AUDIT DURATION

13.07-30.07.2026

LANGUAGE

Solidity

HackenProof | Multyr

TABLE OF CONTENTS

Introduction 3

Audit summary 4

Findings list 4-33

About project 34

HackenProof | Multyr

INTRODUCTION

HackenProof Audit is your solution to maximize cybersecurity through the involvement of the industry's most skilled and experienced community security researchers.

Name	Multyr
Website	https://www.multyr.fi/
Type	Private Security Audit
Date	13.07-30.07.2026
Changelog	30.07.2026 - Preliminary Report 07.08.2026 - Final Report
Scope	https://github.com/hackenproof-public/MTRY
Approved by	HackenProof team
Audited by	@tchkvsky @hellnia

HackenProof | Multyr

AUDIT SUMMARY

7	7	0
Total Findings	Resolved	Informative

Critical	0
High	0
Medium	1
Low	4
None	2

FINDINGS LIST

MEDIUM

MULTYR-5
Global TGE vesting in `PreMTRYClaim` lets late lockers immediately vest and exhaust the release cap — Resolved

LOW

MULTYR-2
Lazy rollover makes GlobalUnlockController and EcosystemEmissionController advertise stale capacity after period boundaries — Resolved

MULTYR-3
GlobalUnlockController's source allowance view omits authorization and genesis preconditions — Resolved

MULTYR-4
VestingManager registry returns the former beneficiary after a supported reassignment — Resolved

MULTYR-6 Immutable guardian in `VestingWalletLinear` lets stale guardians indefinitely freeze non-revocable grants		Resolved

- NONE

MULTYR-1 PreMTRYSale can accept payment after PreMTRYClaim has permanently closed conversion		Resolved
MULTYR-8 Informational		Resolved

VULNERABILITY DETAILS

MULTYR-5 Global TGE vesting in `PreMTRYClaim` lets late lockers immediately vest and exhaust the release cap

• MEDIUM

Target https://github.com/hackenproof-public/MTRY

Status: Resolved

Severity: • MEDIUM

Vulnerability details

[M-1] Global TGE vesting in PreMTRYClaim lets late lockers immediately vest and exhaust the release cap

Risk Category: Release-schedule bypass / temporary fund availability denial

Location:

- PreMTRYClaim.lockPreTokens lines 123-139
- PreMTRYClaim._vestedAmount lines 266-277
- GlobalUnlockController.consumeAllowance lines 242-297

Description: lockPreTokens() converts preMTRY into an MTRY entitlement, but it does not persist any per-user lock timestamp. The only user-specific state written by the lock is the amount of preMTRY locked and the resulting MTRY entitlement:

```
uint256 mtryAmount = amount * CONVERSION_RATIO;

lockedPreAmount[msg.sender] += amount;
totalEntitledMTRY[msg.sender] += mtryAmount;
totalEntitled += mtryAmount;
```

The conversion can be performed until the global end of the claim vesting schedule:

```
if (tgeTimestamp != 0 && block.timestamp >= fullVestingEnd()) {
    revert LockDeadlineExpired();
}
```

Once the entitlement exists, _vestedAmount() calculates vesting from the global TGE timestamp, not from when the holder locked:

```
uint256 vestingStart = tgeTimestamp + CLIFF_DURATION;
uint256 vestingEnd = vestingStart + VESTING_DURATION;
```

A holder can therefore wait until just before fullVestingEnd(), lock preMTRY, and immediately have almost the full 1:4 MTRY entitlement treated as vested. The user skipped the economic effect of the 180-day cliff and nearly all of the 540-day linear release schedule, even though the contract still treats the claim as normal.

The timing issue is amplified by the release controller. PreMTRYClaim is configured as a protected source. Protected sources are allowed to consume the remaining global cap, not only their reserved quota:

```
if (protectedSource[msg.sender]) {
    uint256 protectedRemaining = totalUsed >= cap ? 0 : cap - totalUsed;
    granted = amount <= protectedRemaining ? amount : protectedRemaining;
}
```

All preMTRY holders release through the same PreMTRYClaim source address. The controller does not distinguish Alice's claim backlog from Bob's claim backlog, so whichever holder claims first can consume the period budget for the entire claim contract. Shared-lane vesting wallets and ecosystem emissions then see no remaining allowance for that period.

Impact: The caller cannot exceed their own backed entitlement and cannot inflate supply. The impact is accelerated release and temporary denial of the period release budget to other beneficiaries. With the default 2% cap over the 300M reference supply, the period allowance is 6M MTRY. A max 40M claim entitlement can therefore monopolize roughly seven 30-day periods.

Proof of Concept: Run:

```
forge test --match-path test/AuditFindings.t.sol --match-test "test_M1" -vv
```

Expected:

- test_M1_LateLockDrainsCapAndBlocksSharedVesting shows a late locker consuming the full 6M period cap and causing a vested wallet claim to revert with UnlockThrottled.
- test_M1_LateWhaleBlocksAnotherClaimHolder shows the same protected-source bottleneck blocking a second preMTRY holder.

Recommendation:

Close the lock window before cliff vesting starts, or store per-lock vesting anchors. If global-TGE vesting is intentional, isolate the preMTRY claim budget from the shared vesting/emission lane.

```
 function lockPreTokens(uint256 amount) external nonReentrant whenNotPaused {
     if (!initialized) revert NotInitialized();
     if (amount == 0) revert ZeroAmount();
-    if (tgeTimestamp != 0 && block.timestamp >= fullVestingEnd()) revert
LockDeadlineExpired();
+    if (tgeTimestamp != 0 && block.timestamp >= tgeTimestamp + CLIFF_DURATION)
{
+        revert LockDeadlineExpired();
+    }
    // existing transfer, burn, and entitlement accounting follows
 }
```

If late locking must remain supported, record each lock as a separate tranche or use a weighted lock timestamp for the user's entitlement so later deposits cannot inherit an earlier schedule.

Validation steps

M-1. Global TGE vesting in PreMTRYClaim lets late lockers immediately vest and exhaust the release cap

Test file: test/AuditFindings.t.sol

Test function(s):

- test_M1_LateLockDrainsCapAndBlocksSharedVesting
- test_M1_LateWhaleBlocksAnotherClaimHolder

Run both M-1 tests:

```
forge test --match-path test/AuditFindings.t.sol --match-test "test_M1" -vv
```

Run individually:

```
forge test --match-path test/AuditFindings.t.sol --match-test
test_M1_LateLockDrainsCapAndBlocksSharedVesting -vv
forge test --match-path test/AuditFindings.t.sol --match-test
test_M1_LateWhaleBlocksAnotherClaimHolder -vv
```

Validation flow:

1. Alice buys the full preMTRY sale allocation.
2. TGE is set.
3. Alice waits until claim.fullVestingEnd() - 1.
4. Alice locks preMTRY only at the end of the schedule.
5. The test confirms Alice's claimable amount is already near the full 40M MTRY entitlement.
6. Alice claims first and consumes the 6M MTRY period cap.
7. A vesting wallet claim, and separately Bob's preMTRY claim, are throttled.

Expected evidence:

- claim.totalEntitledMTRY(alice) == 40_000_000e18.
- claim.claimable(alice) > 39_900_000e18 immediately after the late lock.
- mtry.balanceOf(alice) == unlockController.maxUnlockForPeriod(period).
- Later valid claims revert with UnlockThrottled.

PoC test functions:

```
function test_M1_LateLockDrainsCapAndBlocksSharedVesting() public {
    vm.warp(saleStart);
    _fundAlice(FULL_SALE_USDC);
    vm.prank(alice);
    sale.buy(FULL_SALE_USDC, alice);

    _setTGE(block.timestamp);

    VestingWalletLinear victimWallet = VestingWalletLinear(
        _createGrantVesting(VestingManager.BucketType.Team, teamBeneficiary,
10_000_000e18, 0, 1 days, false)
    );
```

```
    vm.warp(claim.fullVestingEnd() - 1);

    vm.startPrank(alice);
    preMTRY.approve(address(claim), FULL_PRE_SUPPLY);
    claim.lockPreTokens(FULL_PRE_SUPPLY);
    vm.stopPrank();

    assertEq(claim.totalEntitledMTRY(alice), FULL_ENTITLEMENT);
    assertGt(claim.claimable(alice), 39_900_000e18);
    assertGt(victimWallet.releasableAmount(), 0);

    uint256 period = unlockController.currentPeriod();
    uint256 cap = unlockController.maxUnlockForPeriod(period);
    assertEq(cap, 6_000_000e18);

    vm.prank(alice);
    claim.claim();

    assertEq(mtry.balanceOf(alice), cap);
    assertEq(unlockController.totalReleasedInPeriod(period), cap);
    assertEq(unlockController.remainingAllowanceForSource(address(victimWallet)),
0);

    vm.prank(teamBeneficiary);
    vm.expectRevert(UnlockThrottled.selector);
    victimWallet.claim();
}

function test_M1_LateWhaleBlocksAnotherClaimHolder() public {
    vm.warp(saleStart);
    _fundAlice(FULL_SALE_USDC);
    vm.prank(alice);
    sale.buy(FULL_SALE_USDC, alice);

    _setTGE(block.timestamp);

    uint256 bobPre = 1_000_000e18;
    uint256 alicePre = FULL_PRE_SUPPLY - bobPre;
    vm.prank(alice);
    assertTrue(preMTRY.transfer(bob, bobPre));

    vm.warp(claim.fullVestingEnd() - 1);

    vm.startPrank(alice);
    preMTRY.approve(address(claim), alicePre);
    claim.lockPreTokens(alicePre);
    vm.stopPrank();

    vm.startPrank(bob);
    preMTRY.approve(address(claim), bobPre);
    claim.lockPreTokens(bobPre);
    vm.stopPrank();
```

```
    uint256 period = unlockController.currentPeriod();
    uint256 cap = unlockController.maxUnlockForPeriod(period);
    assertGt(claim.claimable(alice), cap);
    assertGt(claim.claimable(bob), 0);

    vm.prank(alice);
    claim.claim();
    assertEq(mtry.balanceOf(alice), cap);

    vm.prank(bob);
    vm.expectRevert(UnlockThrottled.selector);
    claim.claim();
    assertEq(mtry.balanceOf(bob), 0);
}
```

VULNERABILITY DETAILS

MULTYR-2 Lazy rollover makes GlobalUnlockController and EcosystemEmissionController advertise stale capacity after period boundaries

• LOW

Target https://github.com/hackenproof-public/MTRY

Status: Resolved

Severity: • LOW

Vulnerability details

Hi Multyr team. Your code is really well hardened. Your fuzzing and formal verification is really making my work difficult.

Summary: Pending release-cap changes become effective in the next period or epoch, but are promoted only inside state-changing consumption functions. The public capacity views do not simulate this due rollover. Immediately after a boundary, they fall back to the old current cap, while the next state-changing call at the same timestamp first promotes the pending cap and enforces a different value.

This produces deterministic view/execution disagreement in both controllers. More importantly, GlobalUnlockController.maxUnlockForPeriod() expressly promises that forward projections use pending values to preview the upcoming cap. Its two inputs fail in opposite directions: the cap branch ignores an already-due pending cap, while the reference-supply branch immediately uses a value committed in the current period even though the documented commit rule says that value applies only to strictly later periods. This is a documented projection-contract violation, not merely an objection to lazy state promotion.

Reviewed commit: ef6e11251ff240ed5315f410b2b29c24b6fd007c.

Details: Global unlock reproduction:

1. The current cap is 20% of a 1,000,000-token reference supply: 200,000 tokens.
2. Governance schedules a reduction to 10% during period 0.
3. Time advances into period 1 without a consuming transaction.
4. maxUnlockForPeriod(1) and remainingAllowance() both return the old 200,000-token capacity.
5. The immediately following consumeAllowance(200,000) rolls period 1 and grants only 100,000.

Ecosystem reproduction:

1. The current local epoch cap is 1,000 tokens.
2. Governance schedules a reduction to 500 during epoch 0.
3. Time advances into epoch 1 without an emission.
4. availableThisEpoch() returns the old 1,000-token capacity.
5. The immediately following emitTo(..., 1,000) rolls epoch 1 and grants only 500.

Code excerpts: src/GlobalUnlockController.sol:

```
/// @dev ... Forward projections for periods not yet initialized use current pending
values ...
function maxUnlockForPeriod(uint256 period) public view returns (uint256) {
    uint256 capBps = capBpsByPeriod[period];
    // ...
    if (capBps == 0) {
        capBps = currentUnlockCapBps;
    }
    // ...
}

function _rollPeriodIfNeeded(uint256 period) internal {
    // ...
    if (pendingCapCommitPeriod < p && pendingUnlockCapBps !=
currentUnlockCapBps) {
        currentUnlockCapBps = pendingUnlockCapBps;
    }
    capBpsByPeriod[p] = currentUnlockCapBps;
    // ...
}
```

The two fallback branches are inconsistent with rollover in complementary states:

- When a pending cap change is already due, the view uses currentUnlockCapBps, while rollover promotes and snapshots pendingUnlockCapBps because pendingCapCommitPeriod < period.
- When governance calls setReferenceSupply() after entering an uninitialized current period, the setter immediately overwrites the single live referenceSupply slot and records referenceSupplyCommitPeriod == period. The view therefore uses the new value immediately. An immediate consuming call that completes rollover instead takes the referenceSupplyCommitPeriod < p false branch and snapshots the prior period's old reference supply. The view has projected a not-yet-due value.

The reference branch is correct only when the live reference change is already due for the queried period. It does not distinguish a due value from a same-period pending value.

src/EcosystemEmissionController.sol:

```
function availableThisEpoch() external view returns (uint256) {
    // ...
    uint256 cap = maxEmissionByEpoch[epoch];
    if (cap == 0) cap = currentMaxEmissionPerEpoch;
    // ...
}

function _emit(address recipient, uint256 amount) internal returns (uint256 granted)
{
    // ...
    _rollEpochIfNeeded(epoch);
    uint256 epochCap = maxEmissionByEpoch[epoch];
    // ...
}

function _rollEpochIfNeeded(uint256 epoch) internal {
    // ...
    if (pendingMaxCommitEpoch < e && pendingMaxEmissionPerEpoch !=
currentMaxEmissionPerEpoch) {
        currentMaxEmissionPerEpoch = pendingMaxEmissionPerEpoch;
    }
    maxEmissionByEpoch[e] = currentMaxEmissionPerEpoch;
    // ...
}
```

Impact: Off-chain integrations, indexers, and monitoring tooling cannot rely on the primary capacity views around lazy boundaries and same-period parameter commits.

Repository audit material describes dashboards reading allowance views and availableThisEpoch() as an operator preflight guard. A due cap reduction makes the views overstate capacity; a due cap increase makes them understate it. A same-period reference reduction makes the view prematurely understate current-period capacity, while an increase prematurely overstates it. An operator sizing a transaction from the stale view can receive a different clamped amount than quoted or incorrectly defer available capacity.

To me, this is Low under functional-correctness scope. The documented capacity projection advertises a result the next transaction at the same timestamp will not deliver, but state-changing paths enforce the intended cap.

Mitigations: Make views compute the cap that would be promoted if rollover occurred at the current timestamp:

- For an uninitialized GlobalUnlockController period, select the pending cap when pendingCapCommitPeriod < period; apply the equivalent projected logic for protected quota and reference supply where relevant.
- For an uninitialized ecosystem epoch, select pendingMaxEmissionPerEpoch when pendingMaxCommitEpoch < epoch.
- Prefer internal pure/view helpers shared by projection and rollover so the two paths cannot diverge.
- Add boundary tests that call the view before the first state-changing transaction after both cap increases and decreases.

Do not call a state-changing rollover merely to read capacity; the view should simulate the deterministic promotion without storage writes.

Validation steps

PoC overview:

I built 4 Foundry tests to cover both directions in both controllers.
They:

- schedule a cap reduction or increase
- move into the effective period/epoch without triggering rollover
- assert the stale view
- then execute the consuming operation at the same timestamp and assert the promoted cap.

There is also another source test covering the inverse reference-supply case: governance commits a reference reduction after entering period 1, the view prematurely reports the lower cap, and an immediate write snapshots and enforces the old reference for the commit period.

How to run:

From the MTRY/ repository root:

```
forge test \
  --match-path test/security/BusinessLogicPoCs.t.sol \
  --match-test 'test_(globalUnlock|ecosystem)View(UsesStale|
UnderstatesPending)Cap' \
  -vvv
```

If the installed Foundry version does not interpret the expression as a regex, run the complete file:

```
forge test --match-path test/security/BusinessLogicPoCs.t.sol -vv
```

Code:

```
function test_globalUnlockViewUsesStaleCapUntilFirstStateChangingCall() public {
    unlockController.setAuthorizedSource(address(this), true);
    unlockController.setProtectedSource(address(this), true);

    unlockController.setPendingUnlockCapBps(1_000);
    vm.warp(unlockController.genesisTimestamp() +
unlockController.periodDuration());

    assertEq(unlockController.maxUnlockForPeriod(1), 200_000e18);
    assertEq(unlockController.remainingAllowance(), 200_000e18);

    uint256 granted = unlockController.consumeAllowance(200_000e18);
    assertEq(granted, 100_000e18);
    assertEq(unlockController.maxUnlockForPeriod(1), 100_000e18);
}

function test_ecosystemViewUsesStaleCapUntilFirstStateChangingCall() public {
    EcosystemEmissionController ecosystem = new EcosystemEmissionController(
        address(token), address(unlockController), address(this),
        block.timestamp, 30 days, 1_000e18
    );

    token.mint(address(ecosystem), 10_000e18);
    ecosystem.setApprovedRecipient(ALICE, true);
    unlockController.setAuthorizedSource(address(ecosystem), true);
    unlockController.setProtectedSource(address(ecosystem), true);
```

```
    ecosystem.setPendingMaxEmissionPerEpoch(500e18);
    vm.warp(ecosystem.genesis() + ecosystem.epochDuration());

    assertEq(ecosystem.availableThisEpoch(), 1_000e18);

    uint256 granted = ecosystem.emitTo(ALICE, 1_000e18);
    assertEq(granted, 500e18);
    assertEq(ecosystem.availableThisEpoch(), 0);
}
```

The complete harness should be placed in test/security/BusinessLogicPoCs.t.sol.

I added another PoC coming from my own fuzzing. To run it:

```
git clone https://github.com/hackenproof-public/MTRY.git
cd MTRY
git checkout ef6e11251ff240ed5315f410b2b29c24b6fd007c
git submodule update --init --recursive

mkdir -p test/fuzz
cp ~/Downloads/DeepReferenceSupplyCommitFuzz.t.sol test/fuzz/
DeepReferenceSupplyCommitFuzz.t.sol
```

```
forge test \
  --match-contract DeepReferenceSupplyCommitFuzzTest \
  --match-test
test_samePeriodReferenceCommitCreatesViewWriteDisagreementWithoutDorma
ncy \
  -vv
```

Expected output:

```
[PASS] test_ecosystemViewUsesStaleCapUntilFirstStateChangingCall()
[PASS] test_ecosystemViewUnderstatesPendingCapIncreaseUntilRollover()
[PASS] test_globalUnlockViewUsesStaleCapUntilFirstStateChangingCall()
[PASS] test_globalUnlockViewUnderstatesPendingCapIncreaseUntilRollover()
Suite result: ok. 4 passed; 0 failed; 0 skipped
```

VULNERABILITY DETAILS

MULTYR-3 GlobalUnlockController's source allowance view omits authorization and genesis preconditions

• LOW

Target https://github.com/hackenproof-public/MTRY

Status: Resolved

Severity: • LOW

Vulnerability details

3rd finding out of 4. Thank you for reading.

Summary: remainingAllowanceForSource(source) calculates lane capacity without mirroring two preconditions enforced by consumeAllowance(): source authorization and unlock genesis. An arbitrary unauthorized address therefore receives a positive quote even though its corresponding write always reverts. Before genesis, even an authorized protected source receives a full-cap quote while both the aggregate view and consumption path return zero.

Reviewed commit: ef6e11251ff240ed5315f410b2b29c24b6fd007c.

Details: The function is specifically documented as the remaining allowance "for a specific source". Its output is not actionable for an address that cannot consume allowance or before consumption has started.

The pre-genesis facet is particularly well evidenced by your remediation history. internal/AUDIT_RESPONSE_ROUND.md records that POC-06 fixed the aggregate remainingAllowance() view by adding a pre-genesis zero guard, and then instructs operators to use remainingAllowanceForSource(addr) for source-specific capacity. The same guard was not added to that recommended per-source view. This is a live residual of the incomplete aggregate-view remediation, not a claim that the already-fixed function is still vulnerable.

Code excerpts:

```
function remainingAllowanceForSource(address source) external view returns
(uint256) {
    // Missing: if (block.timestamp < genesisTimestamp) return 0;
    // Missing: if (!authorizedSource[source]) return 0;
    uint256 period = currentPeriod();
    uint256 cap = maxUnlockForPeriod(period);
    // ...returns protected or shared capacity...
}

function consumeAllowance(uint256 amount) external whenNotPaused returns
(uint256 granted) {
    if (!authorizedSource[msg.sender]) revert UnauthorizedSource();
    if (block.timestamp < genesisTimestamp) return 0;
}
```

Impact: Integrations and monitoring tools can treat an unauthorized address as an active release source, or report capacity before releases have begun.
In both states the view advertises capacity no corresponding transaction can consume.

Enforcement is not bypassed and no funds are directly lost.
Since the scope accepts functional correctness even without direct fund impact, I am submitting this as a Low finding.

Mitigations: Return zero before genesis and for unauthorized sources, or
revert UnauthorizedSource for the latter so the view mirrors the state-changing path.
Add tests asserting view/write consistency across authorization and time states.

Validation steps

PoC overview: Two tests prove both of the missing preconditions:

1. An unauthorized address receives a positive view result and its corresponding write reverts
2. Before genesis, an authorized protected source receives a 6,000,000 MTRY view result while remainingAllowance() and consumeAllowance() return zero

How to run:

```
cp ~/Downloads/LifecycleInvariantPoCs.t.sol MTRY/test/security/
LifecycleInvariantPoCs.t.sol
cp ~/Downloads/UnlockAllowanceEdgeCases.t.sol MTRY/test/security/
UnlockAllowanceEdgeCases.t.sol
forge test --match-path test/security/LifecycleInvariantPoCs.t.sol --match-test
test_unauthorizedSourceGetsPositiveAllowanceView -vv
forge test --match-path test/security/UnlockAllowanceEdgeCases.t.sol --match-test
test_probe_remainingAllowanceForSourcePreGenesis -vv
```

Code: See:

- test/security/
  LifecycleInvariantPoCs.t.sol::test_unauthorizedSourceGetsPositiveAllowanceView
- test/security/
  UnlockAllowanceEdgeCases.t.sol::test_probe_remainingAllowanceForSourcePreGenesis

Expected output:

```
[PASS] test_unauthorizedSourceGetsPositiveAllowanceView()
[PASS] test_probe_remainingAllowanceForSourcePreGenesis()
Combined result: 2 passed; 0 failed; 0 skipped
```

VULNERABILITY DETAILS

MULTYR-4 VestingManager registry returns the former beneficiary after a supported reassignment

• LOW

Target https://github.com/hackenproof-public/MTRY

Status: Resolved

Severity: • LOW

Vulnerability details

This is my last finding at the moment. I am not giving up and will continuing working on your project, but this is a really hardened codebase.
Thank you.

Summary:

VestingWalletLinear supports a delayed beneficiary reassignment, but confirmation updates only the child wallet. The corresponding VestingManager.VestingRecord.beneficiary is never synchronized.

The manager's documented on-chain "registry for per-beneficiary vesting wallets" permanently returns the creation beneficiary instead of the current beneficiary.
I tried to find a fix bypass and found a corroborating instance of the same missing wallet-to-manager synchronization: VestingRecord.active also remains true after the wallet and manager have both recorded a completed revocation.
I am including as added evidence, just so you know i.

Reviewed commit: ef6e11251ff240ed5315f410b2b29c24b6fd007c.

Details:

1. Governance creates a grant for Alice and the manager stores Alice in its record.
2. Alice proposes Bob through the supported two-phase update.
3. After the delay, Bob confirms.
4. wallet.beneficiary() correctly returns Bob, who now controls claims.
5. getVestingsByBucketPaginated() still returns Alice for the same wallet and has no indicator that this field is historical.

The same synchronization omission affects revocation:

1. Governance creates a revocable grant: the manager stores active: true.
2. The live manager owner calls the wallet's supported revokeUnvested() path.
3. The wallet sets revoked = true, transfers the unvested amount, and successfully calls VestingManager.recordRevocation().
4. The manager records the revocation for bucket accounting, but its public VestingRecord.active field remains true. This is also true for an immediately revoked wallet with no remaining vested entitlement.

Code excerpts:

```
// VestingManager._registerRecord
vestings[id] = VestingRecord({
    // ...
    vestingWallet: vestingWallet,
    beneficiary: p.beneficiary,
    // ...
});

// VestingWalletLinear.confirmBeneficiaryUpdate
address old = beneficiary;
beneficiary = pendingBeneficiary;
pendingBeneficiary = address(0);
emit BeneficiaryUpdateConfirmed(old, beneficiary);

// VestingWalletLinear._doRevoke
revoked = true;
revokedAt = block.timestamp;
IVestingManager(vestingManager).recordRevocation(address(this), unvested);

// VestingManager.recordRevocation updates accounting, but not the record's
lifecycle flag
walletRevocationRecorded[wallet] = true;
VestingRecord storage record = vestings[walletRecordId[wallet]];
totalBucketRevoked[_capBucketFor(record.bucket)] += returnedToTreasury;
// Missing: record.active = false (or an explicitly documented status model)
```

Beneficiary confirmation has no manager callback. Revocation does have an authenticated manager callback, but that callback updates only bucket accounting. Neither the public mapping getter nor the paginated registry view dynamically resolves the wallet's authoritative beneficiary() or revoked() state.

Impact:

Indexers, governance tooling, and users relying on the manager registry can misattribute a wallet and its allocation after a normal ownership transfer, and can continue treating a completed revoked grant as active.
Claim and revocation enforcement remain correct in the wallet, so I can't demonstrate theft of funds.
Since the scope accepts functional correctness, I am adding this as a Low.

Mitigations:

- Treat the wallet as the source of truth and dynamically read beneficiary() and revoked() when returning records
- or add authenticated manager callbacks that synchronize beneficiary on confirmation and active on revocation
- if historical semantics are intended, rename the fields to initialBeneficiary and an unambiguous status such as registered, rather than presenting them as current beneficiary and active state.

Validation steps

PoC overview:

My first test does the complete two-day reassignment and compares the authoritative wallet beneficiary with the manager's returned record.
My second test completes an authenticated revocation and compares wallet.revoked() and walletRevocationRecorded() with the manager record's still-true active flag.

How to run:

```
cp ~/Downloads/LifecycleInvariantPoCs.t.sol test/security/LifecycleInvariantPoCs.t.sol
forge test \
  --match-path test/security/LifecycleInvariantPoCs.t.sol \
  --match-test 'test_(beneficiaryReassignmentLeavesManagerRegistryStale|
revocationLeavesManagerRegistryMarkedActive)' \
  -vv
```

Code:

See:

- test/security/LifecycleInvariantPoCs.t.sol::test_beneficiaryReassignmentLeavesManagerRegistryStale
- test/security/LifecycleInvariantPoCs.t.sol::test_revocationLeavesManagerRegistryMarkedActive

Expected output:

```
[PASS] test_beneficiaryReassignmentLeavesManagerRegistryStale()
[PASS] test_revocationLeavesManagerRegistryMarkedActive()
Suite result: ok. 2 passed; 0 failed; 0 skipped
```

Thank you for reading my submissions!

VULNERABILITY DETAILS

MULTYR-6 Immutable guardian in `VestingWalletLinear` lets stale guardians indefinitely freeze non-revocable grants

• LOW

Target https://github.com/hackenproof-public/MTRY

Status: Resolved

Severity: • LOW

Vulnerability details

[L-1] Immutable guardian in VestingWalletLinear lets stale guardians indefinitely freeze non-revocable grants

Risk Category: Temporary or permanent freeze of vested funds

Location:

- VestingWalletLinear.guardian line 67
- VestingWalletLinear._doRevoke lines 193-215
- VestingWalletLinear.pause / unpause lines 281-288
- VestingManager.vestingGuardian line 104
- VestingManager._deployWallet lines 309-322
- GovernanceRoles.setGuardian lines 45-50

Description: Each VestingWalletLinear stores its guardian as an immutable constructor value:

```
address public immutable guardian;
```

The wallet pause path authorizes either that immutable guardian or the dynamic revoker. The unpause path is revoker-only:

```
function pause() external {
    if (msg.sender != guardian && msg.sender != revoker()) revert NotAuthorized();
    _pause();
}

function unpause() external onlyRevoker {
    _unpause();
}
```

VestingManager passes vestingGuardian into every new wallet:

```
return address(new VestingWalletLinear(
    token,
    p.beneficiary,
    startTs,
    // cliff, duration, allocation, revocability, manager, treasury, controller
    vestingGuardian
));
```

The manager also inherits a mutable guardian field from GovernanceRoles, but that is a separate manager-level role:

```
function setGuardian(address newGuardian) external onlyOwner {
    guardian = newGuardian;
}
```

Calling VestingManager.setGuardian() rotates the manager's own guardian but does not change vestingGuardian and does not reach already-deployed wallets. There is also no wallet-level guardian setter.

A compromised or rogue wallet guardian can therefore keep authority over every wallet it was originally assigned to, even after governance performs the expected manager-level rotation. The owner can unpause a wallet, but the old guardian can immediately pause it again. For non-revocable wallets, emergencyRevokeUnvested() is unavailable, so governance cannot migrate the locked position through the existing emergency path.

Impact: Beneficiary claim() and claimTo() calls can be durably frozen. For revocable grants, governance can claw back the unvested amount and re-grant, but the vested unclaimed portion remains frozen until the pause loop is resolved. For non-revocable grants, the full grant can become permanently inaccessible through the current wallet.

Proof of Concept: Run:

```
forge test --match-path test/AuditFindings.t.sol --match-test
test_L1_RotatedOutGuardianCanDurablyFreezeNonRevocableWallet -vv
```

Expected: manager-level guardian rotation succeeds, but the deployed wallet still recognizes the old guardian. The old guardian pauses the wallet repeatedly, the rotated-in guardian cannot pause/unpause the wallet, and non-revocable emergency revocation reverts with NotRevocable.

Recommendation:

Resolve the wallet guardian dynamically from the manager, mirroring the existing revoker() pattern. This makes manager-level guardian rotation effective for existing wallets.

```
 interface IVestingManager {
+    function guardian() external view returns (address);
     function owner() external view returns (address);
     function recordRevocation(address wallet, uint256 returnedToTreasury) external;
 }
```

```
-address public immutable guardian;
+function guardian() public view returns (address) {
+    return IVestingManager(vestingManager).guardian();
+}

 function pause() external {
-    if (msg.sender != guardian && msg.sender != revoker()) revert NotAuthorized();
+    if (msg.sender != guardian() && msg.sender != revoker()) revert NotAuthorized();
     _pause();
 }
```

Validation steps

L-1. Immutable guardian in VestingWalletLinear lets stale guardians indefinitely freeze non-revocable grants

Test file: test/AuditFindings.t.sol

Test function(s):

- test_L1_RotatedOutGuardianCanDurablyFreezeNonRevocableWallet

Run:

```
forge test --match-path test/AuditFindings.t.sol --match-test
test_L1_RotatedOutGuardianCanDurablyFreezeNonRevocableWallet -vv
```

Validation flow:

1. A non-revocable vesting wallet is created.
2. The beneficiary claims once to prove the wallet is otherwise functional.
3. Governance rotates the manager guardian.
4. The test confirms the wallet guardian remains the old immutable guardian.
5. The rotated-in guardian cannot pause the wallet.
6. The rotated-out guardian pauses, the beneficiary's claim reverts, and the old guardian can re-pause after owner unpauses.
7. Non-revocable emergency recovery reverts.

Expected evidence:

- wallet.guardian() == compromisedGuardian.
- vestingManager.guardian() == rotatedInGuardian.
- Beneficiary claim reverts with Pausable.EnforcedPause.
- wallet.emergencyRevokeUnvested() reverts with NotRevocable.

PoC test function:

```
function test_L1_RotatedOutGuardianCanDurablyFreezeNonRevocableWallet()
public {
    address compromisedGuardian = safeGuardian;
    address rotatedInGuardian = address(0xA11A);
    address beneficiary = address(0x1111);
```

```
    VestingWalletLinear wallet = VestingWalletLinear(
        _createGrantVesting(VestingManager.BucketType.Team, beneficiary,
1_000_000e18, 0, 365 days, false)
    );
    vm.warp(block.timestamp + 183 days);

    vm.prank(beneficiary);
    wallet.claim();
    assertGt(wallet.claimedAmount(), 0);

    vm.prank(deployer);
    vestingManager.setGuardian(rotatedInGuardian);

    assertEq(wallet.guardian(), compromisedGuardian);
    assertEq(vestingManager.guardian(), rotatedInGuardian);

    vm.prank(rotatedInGuardian);
    vm.expectRevert();
    wallet.pause();

    vm.prank(compromisedGuardian);
    wallet.pause();

    vm.prank(beneficiary);
    vm.expectRevert(Pausable.EnforcedPause.selector);
    wallet.claim();

    vm.prank(deployer);
    wallet.unpause();
    vm.prank(compromisedGuardian);
    wallet.pause();

    vm.prank(deployer);
    wallet.unpause();
    vm.prank(compromisedGuardian);
    wallet.pause();
    assertTrue(wallet.paused());

    vm.prank(deployer);
    vm.expectRevert(NotRevocable.selector);
    wallet.emergencyRevokeUnvested();
}
```

VULNERABILITY DETAILS

MULTYR-1 PreMTRYSale can accept payment after PreMTRYClaim has permanently closed conversion

• NONE

Target https://github.com/hackenproof-public/MTRY

Status: Resolved

Severity: • NONE

Vulnerability details

Summary:

The sale end and the preMTRY conversion deadline are configured independently. PreMTRYSale accepts purchases until its immutable saleEnd, while PreMTRYClaim permanently rejects new locks at TGE + 720 days. Neither contract nor the deployment/TGE scripts require the sale to close first.

With an accepted overlapping configuration, a whitelisted buyer follows the normal purchase path, pays USDC directly to treasury, and receives preMTRY in their own wallet. The same preMTRY can never be converted into the promised 4× MTRY entitlement because the lock deadline has passed irreversibly.

Details:

1. If PreMTRYSale is deployed with saleEnd > TGE + 720 days, its constructor only checks saleStart < saleEnd.
2. Bind it through the normal PreMTRYToken.setSaleContract validation and whitelist a buyer.
3. Set the claim TGE.
4. After TGE + 720 days but before saleEnd, the buyer calls buy(usdcAmount, buyer).
5. USDC is transferred irreversibly to treasuryReceiver, and preMTRY is minted to the buyer.
6. lockPreTokens always reverts LockDeadlineExpired from that point onward, so no MTRY entitlement can be created.

Code excerpts:

```
// PreMTRYSale constructor: no relationship to claim TGE/deadline.
if (saleStart_ >= saleEnd_) revert InvalidTimeRange();

// PreMTRYSale.buy
if (block.timestamp > saleEnd) revert SaleEnded();
paymentToken.safeTransferFrom(msg.sender, treasuryReceiver, usdcAmount);
preToken.mint(receiver, preMTRYAmount);

// PreMTRYClaim.lockPreTokens
if (tgeTimestamp != 0 && block.timestamp >= fullVestingEnd()) {
    revert LockDeadlineExpired();
}

function fullVestingEnd() public view returns (uint256) {
    return tgeTimestamp + CLIFF_DURATION + VESTING_DURATION; // TGE + 720 days
}
```

Deploy.s.sol independently reads SALE_START and SALE_END; SetTGE.s.sol later reads TGE_TIMESTAMP. No cross-contract invariant is enforced.

Impact: The buyer loses use of the USDC transferred to treasury and receives an instrument whose intended 1:4 MTRY conversion path has closed permanently. No later timestamp can restore lockPreTokens, and preMTRY is not itself the promised MTRY allocation.

The prerequisite is a trusted deployment/TGE timestamp relationship accepted by all production validation.
I can't demonstrate that an untrusted actor can create that relationship after finalization, and no currently exposed public sale was demonstrated. I tried, really.
The protocol can be finalized into a lifecycle where the documented sale succeeds but the associated conversion functionality cannot be delivered.

I put it in scope under Functional correctness.

Mitigations:

- Enforce saleEnd < claim.fullVestingEnd() before activating TGE, or require TGE_TIMESTAMP >= SALE_END if sales are strictly pre-TGE.
- Add the invariant to deployment and post-deployment checks.
- Prefer a claim-side grace period derived from max(saleEnd, TGE) if post-TGE sales are supported.
- Pause or permanently close the sale automatically when conversion closes.

Validation steps

PoC overview: The primary test recreates the production deployment topology from scratch. It predicts and deploys MTRY, deploys the long sale as the pretoken's first and only sale binding, initializes the real claim pool, and permanently finalizes both the sale and claim bindings before TGE. It then purchases to the buyer after the conversion deadline, proves treasury receives USDC, and proves conversion permanently reverts.

This specifically rules out a test-fixture artifact: the reproduction does not replace a finalized sale and does not depend on post-deployment rebinding. It demonstrates that the production contracts accept the incompatible timestamps during their intended initial deployment/configuration phase and make that state durable through finalization.

How to run:

```
cp ~/Downloads/LifecycleInvariantPoCs.t.sol MTRY/test/security/
LifecycleInvariantPoCs.t.sol
cd MTRY
forge test --match-contract SaleConversionDeadlineTest -vv
```

Expected output:

```
[PASS] test_finalizedInitialDeploymentAcceptsPurchaseAfterConversionClosed()
Suite result: ok. 1 passed; 0 failed; 0 skipped
```

VULNERABILITY DETAILS

MULTYR-8 Informational

• NONE

Target https://github.com/hackenproof-public/MTRY

Status: Resolved

Severity: • NONE

Vulnerability details

[I-1] Code-size checks in VestingWalletLinear beneficiary updates allow unwhitelisted counterfactual contracts

Description: proposeBeneficiaryUpdate() and confirmBeneficiaryUpdate() enforce the beneficiary-contract whitelist only when the candidate already has code:

```
if (newBeneficiary.code.length > 0) {
    if (!
IVestingManager(vestingManager).isWhitelistedBeneficiaryContract(newBeneficiar
y)) {
        revert InvalidBeneficiaryContract();
    }
}
```

The confirmation path repeats the same code.length gate:

```
if (pendingBeneficiary.code.length > 0) {
    if (!
IVestingManager(vestingManager).isWhitelistedBeneficiaryContract(pendingBenefi
ciary)) {
        revert InvalidBeneficiaryContract();
    }
}
```

That distinguishes "EOA" and "contract" by current code size, not by whether the address can later become a contract. A beneficiary can precompute a CREATE2 address, propose it while empty, wait through the beneficiary-update delay, and confirm it while it is still empty. Both checks take the EOA branch. The beneficiary can then deploy code to that same address, producing a live contract beneficiary that was never whitelisted. The impact is limited to beneficiary-policy enforcement: the current beneficiary can move claim authority to an unwhitelisted contract, but cannot steal another
grant. test_I1_CounterfactualBeneficiaryBypassesContractWhitelist demonstrates the path.

Recommendation:

Re-check the beneficiary contract whitelist at claim time, or require contract beneficiaries to be explicitly whitelisted by address before they can receive claim authority.

[I-2] Unbounded TGE timestamps in setTGE() can brick claim and TGE-mode vesting paths

Description:

setTGE() rejects past timestamps but does not bound how far into the future the one-time timestamp may be:

```
if (tgeTimestamp_ < block.timestamp) revert InvalidTimestamp();
tgeTimestamp = tgeTimestamp_;
```

The value cannot be corrected later:

```
if (tgeTimestamp != 0) revert TGEAlreadySet();
```

Claim vesting then adds fixed durations to that timestamp:

```
uint256 vestingStart = tgeTimestamp + CLIFF_DURATION;
uint256 vestingEnd = vestingStart + VESTING_DURATION;
```

TGE-mode vesting wallets have the same arithmetic shape because VestingManager resolves startTimestamp to TGE and the wallet later adds cliff and vesting duration:

```
uint256 start = startTimestamp + cliffDuration;
uint256 end = start + vestingDuration;
```

A timestamp near uint256.max therefore passes the setter but makes later schedule math overflow. The affected operations then revert before any recovery transfer can complete. This requires an owner configuration error, but it can strand the fixed claim pool and funded TGE-mode vesting wallets with no on-chain correction path because TGE can be set only once. test_I2_ExtremeTgeBricksClaimLifecycle and test_I2_ExtremeTgeBricksFundedTgeModeWallet demonstrate both affected paths.

Recommendation:

Bound accepted TGE timestamps to a realistic future window and validate arithmetic headroom before storing the one-time value. Apply the same bound in VestingManager.setTGE().

[I-3] Single-slot pending updates in release controllers can skip scheduled cap reductions

Description:

Unlock cap, protected quota, reference supply, and ecosystem emission cap updates each use a single mutable pending slot. For example, setting a new unlock cap overwrites pendingUnlockCapBps and stores only the current period as the commit period:

```
pendingUnlockCapBps = newCapBps;
pendingCapCommitPeriod = currentPeriod();
```

The pending value is not applied immediately. It is promoted only when a later state-changing release rolls the period:

```
if (pendingCapCommitPeriod < p && pendingUnlockCapBps !=
currentUnlockCapBps) {
    currentUnlockCapBps = pendingUnlockCapBps;
}
capBpsByPeriod[p] = currentUnlockCapBps;
```

The ecosystem emission controller uses the same pattern for per-epoch emission caps:

```
pendingMaxEmissionPerEpoch = newMax;
pendingMaxCommitEpoch = currentEpoch();
```

This creates a skipped-update window. Governance can commit value A in period N - 1, intending it to apply in period N. If no release initializes period N, and governance writes value B during period N, the single pending slot is overwritten. When period N is finally initialized, value B is not eligible because it was committed in period N, and value A is already gone. The period therefore runs with the old active value. If value A was a reduction, releases in that period can exceed the intended limit. test_I3_PendingUnlockCapCommittedForNextPeriodCanBeSkipped and test_I3_PendingEmissionCapCommittedForNextEpochCanBeSkipped demonstrate the skipped updates.

Recommendation:

Store scheduled values by effective period/epoch, or force the current period/epoch to roll before accepting another pending value. Apply this to unlock cap, protected quota, reference supply, and emission cap updates.

[I-4] Split bucket accounting in VestingManager can overstate fundable vesting headroom

Description:

Bucket capacity is tracked separately from the manager's actual MTRY token balance. Revocation sends unvested tokens to Treasury:

```
if (unvested > 0) {
    token.safeTransfer(treasury, unvested);
}

IVestingManager(vestingManager).recordRevocation(address(this), unvested);
```

The manager records that amount as freed bucket capacity:

```
totalBucketRevoked[capBucket] += returnedToTreasury;
```

That makes the bucket appear reusable, but the tokens are no longer held by VestingManager; they are in Treasury. A later replacement grant must still be funded from the manager's token balance.

The burn path creates the same kind of split. It checks and burns the manager's raw balance:

```
uint256 balance = IERC20(token).balanceOf(address(this));
if (amount > balance) revert BurnExceedsBalance();

ERC20Burnable(token).burn(amount);
```

However, it does not reduce any bucket cap or remaining bucket capacity. In both cases, getBucketRemaining() can report grantable headroom that the manager cannot fund. Grant creation can then fail despite apparently sufficient bucket capacity, or a replacement grant can consume funds expected for another bucket. _fundAndVerify() prevents underfunded wallets from being created, so this is an accounting/planning failure rather than direct fund loss. test_I4_RevocationFreesBucketCapWithoutReturningFundingToManager and test_I4_ManagerBurnCreatesUnfundedBucketHeadroom demonstrate both desync paths.

Recommendation:

Track fundable headroom separately from nominal bucket capacity. If revoked capacity should be reusable, keep the backing tokens in VestingManager; otherwise do not credit it as fundable headroom. Limit burns to balance not reserved for future bucket commitments.

[I-5] Direct revocation transfers in VestingWalletLinear bypass unlock caps and burn delays

Description: revokeUnvested() sends unvested MTRY directly from the vesting wallet to Treasury:

```
if (unvested > 0) {
    token.safeTransfer(treasury, unvested);
}
```

This path does not call GlobalUnlockController.consumeAllowance() and does not use the manager burn lifecycle:

```
proposeBurn(amount);
// wait BURN_DELAY
executeBurn();
```

Governance can therefore create a revocable pre-cliff grant and revoke it to move vesting reserve into the uncapped Treasury path. This is different from a beneficiary claim, which must pass through the controller's period allowance before tokens leave a programmatic release contract. Funds remain protocol-owned, so the impact is semantic and operational: the global unlock controller is not a protocol-wide circulation cap once Treasury and revocation routes are considered. test_I5_RevokeMovesUnvestedReserveToTreasuryWithoutUnlockOrBurnDelay confirms the transfer increases Treasury balance without consuming unlock allowance.

Recommendation:

Document the unlock cap as applying only to programmatic claim/vesting/emission releases, or return revoked tokens to VestingManager and move Treasury recoveries through a delayed governance path.

[I-6] Renounceable ownership in paused GlobalUnlockController can permanently halt controller-gated releases

Description:

All programmatic releases eventually call GlobalUnlockController.consumeAllowance(), which is paused-gated:

```
function consumeAllowance(uint256 amount)
    external
    whenNotPaused
    returns (uint256 granted)
```

The controller pause is intentionally broad and can be triggered by the guardian or owner:

```
function pause() external onlyGuardianOrOwner {
    _pause();
}
```

Unpause is owner-only:

```
function unpause() external onlyOwner {
    _unpause();
}
```

Because the controller inherits OpenZeppelin ownership without overriding renounceOwnership(), the owner can renounce while the controller is paused. After that, owner() is address(0), no account satisfies onlyOwner, and no account can call unpause(). The guardian can halt releases but cannot restore them. This is a privileged operational hazard, not an unprivileged exploit; if it occurs, all controller-gated releases remain frozen unless the protocol migrates to new contracts. test_I6_ControllerPausePlusRenouncedOwnershipMakesReleaseFreezePermanent demonstrates the unrecoverable pause state.

Recommendation:

Disable renounceOwnership() on core release contracts, or make it delayed and unavailable while paused.

Validation steps

I-1. Code-size checks in VestingWalletLinear beneficiary updates allow unwhitelisted counterfactual contracts

Run:

```
forge test --match-path test/AuditFindings.t.sol --match-test
test_I1_CounterfactualBeneficiaryBypassesContractWhitelist -vv
```

Expected evidence:

- The predicted CREATE2 address has no code and is not whitelisted during proposal and confirmation.

- The wallet beneficiary becomes the empty predicted address.
- Code is deployed to that address afterward, and the unwhitelisted contract can claim MTRY.

I-2. Unbounded TGE timestamps in setTGE() can brick claim and TGE-mode vesting paths

Run:

```
forge test --match-path test/AuditFindings.t.sol --match-test "test_I2" -vv
```

Expected evidence:

- Claim path operations revert with stdError.arithmeticError.
- TGE-mode wallet claim() and revokeUnvested() revert with stdError.arithmeticError.
- Both one-time TGE setters reject correction with TGEAlreadySet.

I-3. Single-slot pending updates in release controllers can skip scheduled cap reductions

Run:

```
forge test --match-path test/AuditFindings.t.sol --match-test "test_I3" -vv
```

Expected evidence:

- unlockController.capBpsByPeriod(1) == 200, not the previously committed 100.
- unlockController.capBpsByPeriod(2) == 300, proving only the later value survived.
- ecosystem.maxEmissionByEpoch(1) == 1_000_000e18, not the skipped 100_000e18.

I-4. Split bucket accounting in VestingManager can overstate fundable vesting headroom

Run:

```
forge test --match-path test/AuditFindings.t.sol --match-test "test_I4" -vv
```

Expected evidence:

- After revocation, getBucketRemaining(Team) == TEAM_CAP while mtry.balanceOf(address(vestingManager)) == 0.
- Treasury balance increases by the revoked Team allocation.
- After burn, getBucketRemaining(AdvisorsShort) == ADVISORS_SHORT_CAP.
- The final grant creation reverts despite reported bucket headroom.

I-5. Direct revocation transfers in VestingWalletLinear bypass unlock caps and burn delays

Run:

```
forge test --match-path test/AuditFindings.t.sol --match-test
test_I5_RevokeMovesUnvestedReserveToTreasuryWithoutUnlockOrBurnDelay -vv
```

Expected evidence:

- Treasury balance increases by 1_000_000e18.
- unlockController.totalReleasedInPeriod(period) == 0, proving the transfer did not consume unlock allowance.

I-6. Renounceable ownership in paused GlobalUnlockController can permanently halt controller-gated releases

Run:

```
forge test --match-path test/AuditFindings.t.sol --match-test
test_I6_ControllerPausePlusRenouncedOwnershipMakesReleaseFreezePermanent -
vv
```

Expected evidence:

- unlockController.paused() == true.
- unlockController.owner() == address(0).
- unlockController.unpause() reverts.
- consumeAllowance(1) reverts with Pausable.EnforcedPause.

ABOUT PROJECT

Multyr is a non-custodial capital allocation protocol. It provides ERC-4626 vaults that deploy allocator capital across DeFi strategies within encoded on-chain constraints.

DISCLAIMER

HackenProof Disclaimer

The application provided for assessment has been analyzed in accordance with generally accepted industry practices applicable at the time this report was prepared. The assessment focused on identifying cybersecurity vulnerabilities and security issues in the application's

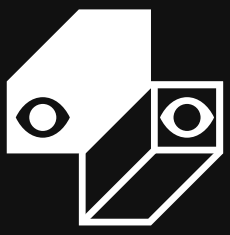
source code, deployment, configuration, and functionality, based solely on the materials, access, and scope made available for the engagement.

This report does not provide any representation or warranty that all vulnerabilities, weaknesses, or misconfigurations have been identified, nor does it guarantee the absolute security, reliability, or bug-free status of the application. The conclusions herein apply only to the specific version and scope reviewed and may no longer remain valid after any modification, update, or environmental change. This report should not be regarded as a definitive or exhaustive statement of the application's security posture.

Technical Disclaimer

Applications, whether decentralized applications (DApps) or other software systems, are deployed and operated within technical environments that may include blockchains, cloud services, operating systems, programming languages, frameworks, libraries, APIs, third-party services, and infrastructure components. Any of these elements may contain inherent weaknesses, undisclosed vulnerabilities, or configuration issues that can result in security incidents, operational failures, or unauthorized access.

The assessment reflects the condition of the application only at the time of testing and within the agreed scope, and the absence of findings in this report must not be interpreted as evidence that no vulnerabilities exist. Continuous security practices, including secure development, independent reviews, monitoring, patch management, and periodic reassessments, remain necessary to maintain an appropriate level of security.



HackenProof

Expert crowdsourced security provider

PRIVATE SECURITY AUDIT

Contact us:

Linkedin | X | Website | Mail

2026